B13 Interpretability and fairness

Generated from the same content files as the app. Each module lists its roadmap months. Solutions are in the appendix.

Contents

Track B: Python for data science 2
B13 Interpretability and fairness 3
B13-L1 Permutation importance on held-out data 3
B13-L2 SHAP values, and what they do not tell you 7
B13-L3 Performance by subgroup, and the fairness trade-offs 11
Track B best-practice checklist 16
Appendix: solutions and answer keys 17

Track B: Python for data science

Write Python and run analyses that are correct, reproducible and easy to review.

B13 Interpretability and fairness

Roadmap: Prerequisite for all months. Permutation importance, SHAP and subgroup performance.

B13-L1 Permutation importance on held-out data

Concept

"Which features matter?" has a trustworthy answer and a misleading one.

The misleading one is **impurity importance**, `feature_importances_` on scikit-learn tree ensembles. It is computed on training data and favours features with many distinct values. In this course's build (scikit-learn 1.9, random forest, 3,000 rows), a meaningless column of random IDs with 3,000 distinct values got an impurity importance of **0.335**, half as much as the real signal (0.661).

The trustworthy one is **permutation importance** on **held-out** data: shuffle one column, breaking its link with the target, and measure how much the score drops. A feature the model relies on causes a large drop. The same random ID column dropped the test AUC by **0.004**: nothing.

```
from sklearn.inspection import permutation_importance
result = permutation_importance(model, X_test, y_test, scoring="roc_auc",
                                n_repeats=10, random_state=0)
result.importances_mean, result.importances_std
```

Read it with three cautions:

- **It measures the model, not the world.** A large drop means this model uses the feature. It does not mean the feature causes the outcome.
- **Correlated features share credit.** If `age` and `years_since_birth_cohort` carry the same information, shuffling one leaves the other, and both look unimportant. Group correlated features and permute them together, or report them as a group.
- **Use the spread.** Report the mean drop with its standard deviation across repeats. A drop of 0.003 ± 0.004 is zero.

Use the metric you report for the model. An importance in accuracy on imbalanced data says little (lesson B12-L1).

Why it matters

A retention model's slide showed "customer ID" as the third most important driver, from `feature_importances_`. The product team asked what it meant. It meant nothing: the ID had thousands of distinct values, so trees split on it to memorise training rows. Permutation importance on the test set gave it zero. The slide had already been shown to the client.

Worked example: weak code

```
importances = dict(zip(X_train.columns, model.feature_importances_))
top = sorted(importances, key=importances.get, reverse=True)[:5]
```

1. **Line 1.** Impurity importance from training data: biased towards high-cardinality features.

Worked example: strong code

```
import numpy as np
import pandas as pd
from sklearn.base import ClassifierMixin
from sklearn.metrics import roc_auc_score


def permutation_drop(model: ClassifierMixin, X: pd.DataFrame, y: pd.Series, col: str, rng: np.random.Generator, n_repeats: int) -> float:
    """Mean fall in held-out ROC AUC when col is shuffled."""
    base = roc_auc_score(y, model.predict_proba(X)[:, 1])
    drops = []
    for _ in range(n_repeats):
        shuffled = X.assign(**{col: rng.permutation(X[col].to_numpy())})
        drops.append(base - roc_auc_score(y, model.predict_proba(shuffled)[:, 1]))
    return float(np.mean(drops))
```

1. **Line 9.** Score on held-out rows, with the reported metric.
2. **Line 12.** Breaks the link between one feature and the target, keeping its distribution.

Measure importance by shuffling on held-out data, with the reported metric, and report the spread.

Recap card

- **Impurity importance bias** Favours high-cardinality features, from training data. A random ID scored 0.335 in this lesson's example.
- **Permutation importance** Shuffle a column on held-out data and measure the score drop, with its spread.

- **Correlated features** Share credit. Permute and report them as a group.
- **What importance means** What the model relies on. Not what causes the outcome.

Practice

Exercise 1. Multiple choice (level 1)

A random ID column ranks high in a random forest's `feature_importances_`. What is the likely cause?

- a) IDs predict the outcome.
- b) Impurity importance favours features with many distinct values, measured on training data.
- c) The forest has too few trees.
- d) The ID was scaled incorrectly.

Why?

- r1) Many split points let trees memorise training rows, which impurity importance rewards.
- r2) Random forests cannot handle integers.

Exercise 2. Predict the output (level 2)

`age` and `birth_year` hold the same information. A model uses both. You permute only `age` on held-out data. What happens to its importance?

```
drop_age_alone = 0.01        # measured
drop_both_together = 0.12    # measured when both are shuffled together
print(drop_age_alone < drop_both_together)
```

Exercise 3. Fill the gap (level 2)

Compute permutation importance on the test set, scored by ROC AUC, with 10 repeats.

```
result = permutation_importance(model, _____(1), y_test,
    scoring="roc_auc", n_repeats=10, random_state=0)
print(result.importances_mean, result._____(2))
```

Exercise 4. Write the function (level 3)

Write `permutation_drop(model, X, y, col, rng, n_repeats) -> float`. Compute the ROC AUC of `model.predict_proba(X)[:, 1]`. Then, `n_repeats` times, replace `col` with `rng.permutation(X[col].to_numpy())` in a **copy** of `X`, recompute the AUC, and record the drop. Return the mean drop. Leave `X` unchanged.

Your code must pass these checks:

- X changed. Shuffle a copy, for example with X.assign(...).
- Shuffle with rng.permutation each repeat, and average base AUC minus shuffled AUC.

Shape of the answer. The reference solution defines these names. Write the bodies yourself.

```python
# Mean fall in ROC AUC when col is shuffled.
def permutation_drop(
    model: ClassifierMixin,
    X: pd.DataFrame,
    y: pd.Series,
    col: str,
    rng: np.random.Generator,
    n_repeats: int,
) -> float: ...
```

Starter code:

```python
import numpy as np


def permutation_drop(model, X, y, col, rng, n_repeats):
    return float(model.coef_[0][list(X.columns).index(col)])
```

Exercise 5. Architecture design (level 4)

A client asks for "the top drivers of churn" from your model. Choose what to deliver.

Options: Permutation importance on the test set, with the reported metric, feature_importances_ from training.

1. Importance method
2. tenure_months and contract_start_year are 0.97 correlated
3. Wording on the slide

Sources

- A random ID with 3,000 values got impurity importance 0.335 and permutation importance (ROC AUC, test) 0.004 in a 200-tree random forest. (verified: Run with scikit-learn 1.9.1 in this repository (random_state=0).)

B13-L2 SHAP values, and what they do not tell you

Concept

Permutation importance (lesson B13-L1) ranks features for the whole model. **SHAP values** explain **one prediction**: how much each feature pushed this customer's score up or down from a baseline.

For one row, SHAP values are **additive**:

```
prediction = base_value + sum of the row's SHAP values
```

The base value is the model's average output over the background data. In shap 0.51:

```
import shap
explainer = shap.TreeExplainer(model)
explanation = explainer(X_test)          # shap.Explanation
explanation.values, explanation.base_values
```

Know which **output space** the values live in. For a scikit-learn `GradientBoostingClassifier`, TreeExplainer explains the log-odds (`decision_function`), not the probability. For a `RandomForestClassifier` it explains probabilities, with one set of values per class, so `.values` has shape `(rows, features, classes)`. This course's build checked both additivity identities on shap 0.51 with scikit-learn 1.9. Say which space you report.

Aggregate carefully. The mean **absolute** SHAP value per feature gives a global ranking. A beeswarm plot shows the direction and spread.

What SHAP does **not** tell you:

- **Causes.** A feature with a large SHAP value is one the model uses. Changing it in the real world may do nothing.
- **Robust truths about correlated features.** When two features carry the same information, the credit split between them depends on the model and the explainer's assumptions.
- **Anything about data the model never saw.** Explanations for out-of-range inputs describe extrapolation.

For customer-facing reasons ("your application was declined because..."), SHAP values are a starting point for wording. They are not a legal explanation on their own.

shap is not available in this app's browser runtime. The exercises use plain Python on SHAP outputs. Run shap itself locally.

Why it matters

A pricing team read a SHAP summary showing that `number_of_support_calls` pushed renewal probability **up**. They proposed encouraging customers to call support more. The model had learned that customers who called support were the engaged ones who had already decided to stay. More calls would not create engagement. An experiment on proactive calls later showed no effect on renewals.

Worked example: weak code

```
explanation = shap.TreeExplainer(rf_classifier)(X_test)
top = X_test.columns[np.argsort(explanation.values.mean(axis=0))[-3:]]
print("Top causes of churn:", list(top))
```

1. **Line 2.** Signed means cancel out, and a random forest's values have a class axis that is ignored here.
2. **Line 3.** Model reliance reported as causes.

Worked example: strong code

```
def top_contributions(names: list[str], values: list[float], k: int) -> list[tuple[str, float]]:
    """The k largest contributions to one prediction, by absolute size, with their sign."""
    if len(names) != len(values):
        raise ValueError("one SHAP value per feature is required")
    ranked = sorted(zip(names, values, strict=True), key=lambda pair: abs(pair[1]), reverse=True)
    return ranked[:k]


def check_additivity(base_value: float, values: list[float], prediction: float, tol: float = 1e-6) -> bool:
    """True when base value plus contributions reproduces the model output."""
    return abs(base_value + sum(values) - prediction) <= tol
```

1. **Line 5.** Rank by size, keep the sign for the wording.
2. **Line 11.** A cheap test that you are reading the right output space.

SHAP explains one prediction additively in a stated output space. It describes the model, not the world.

Recap card

- **SHAP additivity** prediction = base_value + sum of the row's SHAP values, in the explainer's output space.

- **Output space for GradientBoostingClassifier in TreeExplainer** Log-odds (decision_function), not probability.
- **Global ranking from SHAP** Mean absolute SHAP value per feature.
- **What SHAP cannot show** Causes, a stable split between correlated features, behaviour outside the training range.

Practice

Exercise 1. Multiple choice (level 1)

A feature has a large positive SHAP value for most churners. What can you conclude?

- a) Increasing the feature causes churn.
- b) The model uses the feature to push churn scores up.
- c) The feature is measured wrongly.
- d) The model is overfitting.

Why?

- r1) SHAP values attribute the model's output. They say nothing about intervening in the world.
- r2) Positive SHAP values are always spurious.

Exercise 2. Predict the output (level 2)

A GradientBoostingClassifier row has base value -1.2 (log-odds) and SHAP values 0.8, 0.5 and -0.3. What does this print?

What you need to know.

Log-odds convert to a probability with the logistic function: `p = 1 / (1 + e^(-log_odds))`. `math.exp(x)` computes `e^x`.

```
import math

1 / (1 + math.exp(-0.0))   # 0.5: log-odds 0 means even odds
```

```
import math

log_odds = -1.2 + 0.8 + 0.5 - 0.3
print(round(log_odds, 2), round(1 / (1 + math.exp(-log_odds)), 2))
```

Exercise 3. Fill the gap (level 2)

Compute SHAP values for a tree model with shap 0.51.

```
explainer = shap._____(1)(model)
explanation = explainer(X_test)
contributions = explanation._____(2)
```

Exercise 4. Write the function (level 3)

Write `top_contributions(names, values, k) -> list[tuple[str, float]]`. Return the `k` pairs `(name, value)` with the largest **absolute** values, largest first, keeping the sign. Raise `ValueError` if `names` and `values` differ in length.

Your code must pass these checks:

- Rank by absolute size, and keep the sign of each value.
- Mismatched lengths must raise ValueError.
- k larger than the number of features returns all of them.

Shape of the answer. The reference solution defines these names. Write the bodies yourself.

```python
# The k largest contributions to one prediction, by absolute size.
def top_contributions(
    names: list[str],
    values: list[float],
    k: int,
) -> list[tuple[str, float]]: ...
```

Starter code:

```python
def top_contributions(names, values, k):
    return sorted(zip(names, values), key=lambda p: p[1], reverse=True)
    [:k]
```

Exercise 5. Architecture design (level 4)

Explain a declined loan application to the customer service team. Choose each part.

Options: State the space (log-odds) and translate the final score to a probability, Show log-odds contributions labelled as percentage points.

1. Output space shown
2. What the explanation lists
3. Wording

Sources

- shap 0.51 TreeExplainer: additivity holds in decision_function space for GradientBoostingClassifier and in probability space, per class, for RandomForestClassifier. (verified: Checked with np.allclose on shap 0.51.0 and scikit-learn 1.9.1 in this repository.)

B13-L3 Performance by subgroup, and the fairness trade-offs

Concept

A model can be accurate overall and fail one group. Before release, compute the key rates **per subgroup**: age band, region, sex, ethnicity where you hold it lawfully, and any group the decision affects differently.

Per group, report:

- **n** and **positives**, so readers can judge the evidence.
- **Selection rate**: the share flagged.
- **True positive rate** (recall): the share of real positives flagged.
- **False positive rate**: the share of real negatives flagged.
- **Precision**, and a calibration check (lesson B12-L2).

Then face a hard fact. When groups have **different base rates**, a calibrated score cannot also give equal false positive and true positive rates, except in trivial cases (Kleinberg, Mullainathan and Raghavan, 2016; Chouldechova, 2017).

A simulation in this course's build: a perfectly calibrated score for two groups with base rates of 30% and 10%, one threshold of 0.3. In both groups, cases scored 0.3 to 0.4 were positive about 34% of the time. Yet group A had a true positive rate of 0.65 and a false positive rate of 0.39. Group B had 0.15 and 0.03.

So choose, with the people accountable for the decision, which property matters most for this use:

- Equal **calibration**: a score means the same risk in every group.
- Equal **true positive rates**: each group's genuine cases are found at the same rate. Suits allocating help.
- Equal **false positive rates**: each group bears the same rate of wrong flags. Suits decisions with a cost to the flagged person.

Write the choice, the reasons and the measured gaps in the model card (lesson B18-L2).

Removing a protected attribute from the features does not remove its influence. Postcode, first name and shopping patterns act as **proxies**. Keep the attribute available for **measuring**, where lawful, even if the model never uses it.

Why it matters

A hardship-support model flagged customers for proactive help. Overall recall was 0.62. By age band, customers over 70 had recall 0.28: they used fewer digital channels, so the features the model relied on were sparse for them. The group most likely to need help was the least likely to be offered it. A per-group table at validation showed the gap in one line.

Worked example: weak code

```
features = [c for c in df.columns if c not in ("age_band", "sex")]
print("recall", recall_score(y_test, flagged))
```

1. **Line 1.** Dropping attributes does not remove proxies, and leaves nothing to measure gaps with.
2. **Line 2.** One overall number hides a group at a fraction of it.

Worked example: strong code

```
import pandas as pd


def group_rates(df: pd.DataFrame, group: str) -> pd.DataFrame:
    """Selection rate, TPR and FPR per group, with counts. Columns y and flagged are 0/1."""
    rows = []
    for name, g in df.groupby(group, sort=True):
        pos, neg = g["y"] == 1, g["y"] == 0
        flag = g["flagged"] == 1
        rows.append({
            group: name,
            "n": len(g),
            "positives": int(pos.sum()),
            "selection_rate": float(flag.mean()),
            "tpr": float((flag & pos).sum() / pos.sum()) if pos.sum() > 0 else None,
            "fpr": float((flag & neg).sum() / neg.sum()) if neg.sum() > 0 else None,
        })
    return pd.DataFrame(rows)
```

1. **Line 7.** The group attribute is used to measure, whether or not the model uses it.
2. **Line 15.** Guarded: a group with no positives has no TPR.

Measure rates per group with counts, choose the fairness property that fits the decision, and document the gaps.

Recap card

- **Rates to report per group** n, positives, selection rate, TPR, FPR, precision, calibration.
- **Impossibility result** With different base rates, calibration and equal error rates cannot all hold (Kleinberg et al. 2016; Chouldechova 2017).
- **Dropping a protected attribute** Proxies keep its influence. Keep it for measurement where lawful.
- **Choosing a fairness property** From the harm: missed help (TPR), wrong flags (FPR), numbers used directly (calibration).

Practice

Exercise 1. Multiple choice (level 1)

You remove `sex` from a model's features. What happens to sex-related disparities?

- a) They disappear.
- b) They can persist through proxies, and you lose the ability to measure them if you discard the column.
- c) They always get worse.
- d) The model becomes calibrated.

Why?

- r1) Other features correlate with sex, and measurement needs the attribute.
- r2) Removing features never changes a model.

Exercise 2. Predict the output (level 2)

What does this print?

```
groups = {"A": {"tp": 65, "pos": 100, "fp": 150, "neg": 390}, "B": {"tp": 15, "pos": 100, "fp": 25, "neg": 900}}
for name, g in groups.items():
    print(name, round(g["tp"] / g["pos"], 2), round(g["fp"] / g["neg"], 2))
```

Exercise 3. Fill the gap (level 2)

Compute one group's true positive rate and false positive rate from boolean Series `flag`, `pos` and `neg`.

```
tpr = (flag & _____(1)).sum() / pos.sum()
fpr = (flag & _____(2)).sum() / neg.sum()
```

Exercise 4. Write the function (level 3)

Write `group_rates(df, group) -> pd.DataFrame`. `df` has 0/1 columns `y` and `flagged`, and a group column. Return one row per group, sorted, with columns `<group>`, `n`, `positives`, `selection_rate`, `tpr` and `fpr`. Use `None` for `tpr` when a group has no positives, and for `fpr` when it has no negatives.

Your code must pass these checks:

- Return the columns in order, one row per group, sorted.
- selection_rate is the share flagged. TPR divides by positives, FPR by negatives.
- A group with no negatives has no FPR: return None.

Shape of the answer. The reference solution defines these names. Write the bodies yourself.

```python
# Selection rate, TPR and FPR per group, with counts.
def group_rates(
    df: pd.DataFrame,
    group: str,
) -> pd.DataFrame: ...
```

Starter code:

```python
import pandas as pd


def group_rates(df, group):
    return df.groupby(group)["flagged"].mean().reset_index()
```

Exercise 5. Architecture design (level 4)

Choose the fairness property to prioritise for each use.

Options: Equal true positive rates, Equal false positive rates, Equal calibration.

1. Offer proactive hardship support
2. Refer claims for a fraud investigation that delays payment
3. Risk scores used directly as probabilities in pricing

Sources

- Calibrated scores with base rates of 30% and 10% at threshold 0.3 gave TPR 0.65 and FPR 0.39 in one group, 0.15 and 0.03 in the other. (verified: Simulated with NumPy 2.4.6 in this repository (Beta(3,7) and Beta(1,9) risks, 200,000 rows each).)
- Calibration within groups and balance of error rates cannot all hold when base rates differ, except in trivial cases. (verified: Kleinberg, Mullainathan

and Raghavan (2016), Inherent Trade-Offs in the Fair Determination of Risk Scores; Chouldechova (2017), Big Data 5(2).)

Track B best-practice checklist

☐ The project has a pyproject.toml, a virtual environment and pinned dependencies.

☐ ruff and mypy pass before every commit.

☐ Functions are pure where possible. Side effects sit at the edges.

☐ Dtypes are explicit. pandas code is vectorised, with no row-wise apply on large frames.

☐ Inputs are validated with pandera or pydantic at every boundary.

☐ Seeds, config and data versions are recorded for every result.

☐ Weighted estimates use the survey weights. Intervals and effect sizes go with every estimate.

☐ Preprocessing sits inside a scikit-learn Pipeline, fitted on training folds only.

☐ The metric matches the decision. Calibration and subgroup performance are checked.

☐ Every experiment is tracked in MLflow.

☐ Tests cover edge cases, including empty inputs and zeros.

☐ Tests use fixtures and parametrize, compare numbers with stated tolerances, isolate I/O, and run with coverage in CI.

☐ Every model has a model card. Every decision has a log entry.

Appendix: solutions and answer keys

Attempt each exercise before you read its solution. Track D answer keys are here too.

B13-L1 Permutation importance on held-out data

B13-L1 exercise 1

Hints. (1) How many split points does a unique ID offer? (2) Which data is impurity importance computed on? (3) Training data.

Answer. b) Impurity importance favours features with many distinct values, measured on training data. Because: r1) Many split points let trees memorise training rows, which impurity importance rewards.

Unique values give trees endless ways to split training rows. Impurity importance counts those splits. Permutation on held-out data does not.

Common wrong answers.

- `a`: Permutation importance on held-out data would show a drop if it did. In this lesson's example it was 0.004.
- `c`: More trees make the bias more stable, not smaller.
- `d`: Trees are not affected by scaling.
- `b|r2`: Right answer, wrong reason. Forests handle integers. Cardinality causes the bias.

B13-L1 exercise 2

Hints. (1) What does the model still have when only age is shuffled? (2) The same information via birth_year. (3) So the single drop is small.

Answer.

```
1 True
```

Correlated features share credit. Permute them as a group to see the importance of the information they carry.

Common wrong answers.

- `False`: With birth_year intact, the model still has the information, so shuffling age alone costs little.

B13-L1 exercise 3

Hints. (1) Held-out rows. (2) The spread across repeats. (3) `X_test` and `importances_std`.

Answer.

```
result = permutation_importance(model, X_test, y_test, scoring="roc_auc",
    n_repeats=10, random_state=0)
print(result.importances_mean, result.importances_std)
```

Held-out rows show what generalises. The standard deviation shows whether a drop is distinguishable from zero.

Common wrong answers.

- g1 = `X_train`: On training data, memorised features look important. Use held-out rows.

B13-L1 exercise 4

Hints. (1) Base AUC first, on the unshuffled data. (2) `X.assign(**{col: rng.permutation(X[col].to_numpy())})` shuffles a copy. (3) Average the drops with np.mean.

Answer.

```
import numpy as np
import pandas as pd
from sklearn.base import ClassifierMixin
from sklearn.metrics import roc_auc_score


def permutation_drop(model: ClassifierMixin, X: pd.DataFrame, y:
    pd.Series, col: str, rng: np.random.Generator, n_repeats: int) ->
    float:
    """Mean fall in ROC AUC when col is shuffled.

    Args:
        model: Fitted classifier with predict_proba.
        X: Held-out features.
        y: Held-out labels.
        col: Column to shuffle.
        rng: Seeded generator.
        n_repeats: Number of shuffles.

    Returns:
        Mean of base AUC minus shuffled AUC.
    """
    base = roc_auc_score(y, model.predict_proba(X)[:, 1])
    drops = []
    for _ in range(n_repeats):
        shuffled = X.assign(**{col: rng.permutation(X[col].to_numpy())})
```

```
        drops.append(base - roc_auc_score(y, model.predict_proba(shuffled)
    [:, 1]))
    return float(np.mean(drops))
```

Shuffling keeps the column's distribution but breaks its link with the target. The fall in AUC is how much the model relied on it.

B13-L1 exercise 5

Hints. (1) Which method uses held-out data? (2) What happens when one of two twins is shuffled? (3) Does reliance imply cause?

Answer.

- Permutation importance on held-out data, with the metric the model is judged on.
- Correlated features grouped, so shared information is not split into two small numbers.
- Words that describe the model, not causes in the world.

Deliver reliance, measured honestly and worded honestly.

Common wrong answers.

- method = `impurity`: Impurity importance favours high-cardinality features and reflects training data.
- correlated = `separate`: Each would look small, because the other carries the same information.
- wording = `cause`: Importance shows what the model uses. Causes need a different design, such as an experiment.

B13-L2 SHAP values, and what they do not tell you

B13-L2 exercise 1

Hints. (1) What does SHAP attribute? (2) The model's output. (3) Not the outcome in the world.

Answer. b) The model uses the feature to push churn scores up. Because: r1) SHAP values attribute the model's output. They say nothing about intervening in the world.

SHAP describes the model's reliance. Test interventions separately.

Common wrong answers.

- `a`: Causal claims need causal evidence, such as an experiment.
- `c`: Nothing in a SHAP value indicates measurement error.

- `d`: Large contributions are normal for useful features.
- `b|r2`: Right answer, wrong reason. The values are real attributions of the model's output.

B13-L2 exercise 2

Hints. (1) Add the base value and all contributions. (2) That is in log-odds. (3) p = 1 / (1 + exp(-log_odds)).

Answer.

```
-0.2 0.45
```

Additivity holds in log-odds for this model. The probability is 0.45 after the logistic transform, and the contributions do not add up in probability space.

Common wrong answers.

- `-0.2 -0.2`: The contributions sum to log-odds. Convert with the logistic function to get a probability.
- `0.8 0.69`: Include the base value: -1.2 + 0.8 + 0.5 - 0.3 = -0.2.

B13-L2 exercise 3

Hints. (1) The explainer for tree ensembles. (2) The attribute with one number per feature per row. (3) `TreeExplainer` and `values`.

Answer.

```
explainer = shap.TreeExplainer(model)
explanation = explainer(X_test)
contributions = explanation.values
```

Calling the explainer returns a `shap.Explanation`. `.values` holds contributions, `.base_values` the baseline.

Common wrong answers.

- g2 = `base_values`: base_values is the starting point. The per-feature contributions are .values.

B13-L2 exercise 4

Hints. (1) Check the lengths first. (2) `sorted(..., key=lambda pair: abs(pair[1]), reverse=True)`. (3) Slice the first k.

Answer.

```
def top_contributions(names: list[str], values: list[float], k: int) ->
    list[tuple[str, float]]:
    """The k largest contributions to one prediction, by absolute size.

    Args:
```

```
        names: Feature names.
        values: SHAP values for one row, same order as names.
        k: Number of pairs to return.

    Returns:
        (name, value) pairs, largest absolute value first, sign kept.

    Raises:
        ValueError: If names and values differ in length.
    """
    if len(names) != len(values):
        raise ValueError("one SHAP value per feature is required")
    ranked = sorted(zip(names, values, strict=True), key=lambda pair:
abs(pair[1]), reverse=True)
    return ranked[:k]
```

A large negative contribution matters as much as a large positive one. Rank by size, report with sign.

B13-L2 exercise 5

Hints. (1) Do log-odds add as percentages? (2) Local or global? (3) Model or world?

Answer.

- Name the output space and convert only the final score.
- Give the local top contributions for this case, not the global ranking.
- Describe the model's reasoning, not a claim about the person.

Local, space-aware and modest wording keeps explanations accurate.

Common wrong answers.

- space = `mixed`: Log-odds contributions do not add up in probability points. Labelling them so is wrong.
- content = `global`: The global ranking says what the model uses in general, not why this case was declined.
- wording = `cause`: SHAP shows what moved the model's score. It is not evidence about the person.

B13-L3 Performance by subgroup, and the fairness trade-offs

B13-L3 exercise 1

Hints. (1) Which other features correlate with sex? (2) How would you measure a gap without the column? (3) Keep it for measurement where lawful.

Answer. b) They can persist through proxies, and you lose the ability to measure them if you discard the column. Because: r1) Other features correlate with sex, and measurement needs the attribute.

Exclusion is not neutrality. Keep the attribute for measuring gaps, and decide separately whether the model uses it.

Common wrong answers.

- `a`: Proxies such as occupation or purchase history can carry the same information.
- `c`: They may change either way. The point is you can no longer see them.
- `d`: Removing a feature says nothing about calibration.
- `b|r2`: Right answer, wrong reason. Removing a feature does change the model, just not necessarily the disparity.

B13-L3 exercise 2

Hints. (1) TPR = tp / positives. (2) FPR = fp / negatives. (3) 150 / 390.

Answer.

```
A 0.65 0.38
B 0.15 0.03
```

Same threshold, very different error rates, because the groups' base rates differ.

Common wrong answers.

- `A 0.65 0.15 B 0.15 0.03`: FPR divides false positives by negatives: 150 / 390 = 0.38.

B13-L3 exercise 3

Hints. (1) True positives are flagged positives. (2) False positives are flagged negatives. (3) `pos` and `neg`.

Answer.

```
tpr = (flag & pos).sum() / pos.sum()
```

```
fpr = (flag & neg).sum() / neg.sum()
```

Both rates condition on the true class: TPR on positives, FPR on negatives.

Common wrong answers.

- g2 = `pos`: False positives are flagged negatives: flag & neg.

B13-L3 exercise 4

Hints. (1) Loop over `df.groupby(group, sort=True)`. (2) Boolean masks: pos, neg and flag. (3) Guard each denominator.

Answer.

```python
import pandas as pd


def group_rates(df: pd.DataFrame, group: str) -> pd.DataFrame:
    """Selection rate, TPR and FPR per group, with counts.

    Args:
        df: Columns y and flagged (0/1) and the group column.
        group: Name of the group column.

    Returns:
        One row per group, sorted by group.
    """
    rows = []
    for name, g in df.groupby(group, sort=True):
        pos, neg = g["y"] == 1, g["y"] == 0
        flag = g["flagged"] == 1
        rows.append(
            {
                group: name,
                "n": len(g),
                "positives": int(pos.sum()),
                "selection_rate": float(flag.mean()),
                "tpr": float((flag & pos).sum() / pos.sum()) if pos.sum() > 0 else None,
                "fpr": float((flag & neg).sum() / neg.sum()) if neg.sum() > 0 else None,
            }
        )
    return pd.DataFrame(rows)
```

Rates conditioned on the true class show who bears which errors. Counts beside them show how much evidence there is.

B13-L3 exercise 5

Hints. (1) Who is harmed, and by which error? (2) Is the score used as a number? (3) Match the property to the harm.

Answer.

- Help should reach genuine cases in every group at the same rate: TPR.
- Wrong referrals harm people, so wrong flags should not fall more on one group: FPR.
- Prices use the probability itself, which must mean the same in every group: calibration.

No single property fits all uses. Pick the one that protects against the harm this decision causes, and document why.

Common wrong answers.

- support = `fpr`: An unneeded offer of help costs little. Missing people who need it is the harm.
- investigate = `tpr`: The harm is to people wrongly referred. Balance false positive rates.
- pricing = `tpr`: Pricing uses probabilities as numbers. They must be calibrated in each group.